

Scaling Tips

1. 📁 Consultar CSVs sin cargar el fichero en memoria

Pandas

```
df = pd.read_csv("data.csv")
```

DuckDB

```
import duckdb
duckdb.query("SELECT * FROM 'data.csv'").df()
```

Sin `read_csv()` ni `DataFrame`. *DuckDB* consulta directamente al disco evitando la carga en memoria del fichero completo

2. 🔗 Join CSVs como en una base de datos (SQL)

```
SELECT a.*, b.info
FROM 'users.csv' a
JOIN 'purchases.csv' b
ON a.user_id = b.user_id
```

DuckDB ejecuta SQL joins contra los datos. Sin preprocesado ni ficheros temporales. No necesito 💰10GB RAM💰😎😎

3. Si hay que usar Pandas 😬... Al menos que sea rápido 🚀

```
import pandas as pd
import duckdb

df = pd.read_csv("bigfile.csv")
result = duckdb.query("SELECT COUNT(*) FROM df WHERE amount > 100").df()
```

DuckDB optimiza el acceso al `DataFrame`. Es del orden de 5 veces más rápido, a veces incluso más. Otro tanto aplica en relación con las consultas de agregados

```
SELECT country, year, SUM(sales)
FROM df
GROUP BY country, year
```

```
-- Infinitamente mejor que:
-- df.groupby(['country', 'year'])['sales'].sum().reset_index()
```

Sin carga ni conversión 💎💎

```
# ERROR: carga de datos en Pandas
big = pd.read_parquet("bigdata.parquet")
summary = big.groupby("category").amount.mean()

# No hay color
summary = con.execute("""
    SELECT category, AVG(amount) AS avg_amount
    FROM 'bigdata.parquet'
    GROUP BY category
""").df()
```

Otra alternativa es usar *Polars* en vez de *Pandas* (ver [DataFrame](#) y [lab](#))

```
import polars as pl
df_pl = pl.read_csv("data.csv")

duckdb.query("SELECT col1, COUNT(*) FROM df_pl GROUP BY col1").df()
```

Independientemente de que los datos estén en memoria, en disco o en alguna otra librería es muy probable que *DuckDB* los maneje directamente, sin requerir costosas conversiones ([Zero ETL](#))

4. SQL + Parquet = Vago pero robusto

La [Evaluación Perezosa](#) (más info en [DataFrames](#) y [lab](#)) es un concepto de programación que se usa a menudo en lenguajes funcionales, donde la evaluación de expresiones se retrasa hasta que sus resultados sean realmente necesarios. Este enfoque ayuda a optimizar la ejecución de los programas mediante la reducción de la carga computacional, permitiendo un procesamiento y asignación eficiente de los recursos. Funciona organizando tareas de computación en unidades más pequeñas y manejables, llamadas *thunks*, que solo se ejecutan cuando se requieren sus resultados. Las características clave son:

- Computación diferida: El cálculo de las expresiones se pospone hasta que los valores sean explícitamente necesarios
- Uso eficiente de los recursos: Reduce el consumo de memoria y la sobrecarga computacional al evitar cálculos innecesarios
- Ejecución de consultas optimizada: optimiza el procesamiento de consultas complejas de múltiples pasos mediante la consolidación de múltiples operaciones en un solo pase de evaluación

```
# pd.read_parquet() killer 🐼  
duckdb.query("SELECT * FROM 'logs_2025.parquet' WHERE status = 'error'").df()
```

Usando *Polars* como mediador

```
import polars as pl  
  
# Lazy load a big parquet  
pl_df = pl.scan_parquet("bigdata.parquet")  
  
# Query it directly with DuckDB  
result = con.execute("""  
    SELECT category, SUM(amount) as total_amount  
    FROM pl_df  
    GROUP BY category  
""").pl()  
  
print(result)
```

5. 🧹 Limpieza de los Pipelines SQL en Python

DuckDB facilita la legibilidad del código de la [ETL](#)

```
q = """  
WITH cleaned AS (  
    SELECT * FROM 'sales.csv'  
    WHERE price IS NOT NULL  
)  
aggregated AS (  
    SELECT region, AVG(price) as avg_price  
    FROM cleaned  
    GROUP BY region  
)  
SELECT * FROM aggregated  
ORDER BY avg_price DESC  
"""  
  
df = duckdb.query(q).df()
```

Que pasen a escena las [vistas temporales](#) 📺

```
con.execute("CREATE VIEW sales AS SELECT * FROM 'sales.parquet'")  
con.execute("CREATE VIEW customers AS SELECT * FROM 'customers.parquet'")  
  
result = con.execute("""  
    SELECT c.region, SUM(s.amount) AS revenue  
    FROM sales s  
    JOIN customers c ON s.customer_id = c.id  
    GROUP BY c.region  
""").df()
```

Legible. Mantenible. Reproducible 💰💎💰

Tabla Resumen

Comparativa			Bad Pandas
Task	Pandas	DuckDB	
Read CSV	✅ Slow for large files	⚡ Query directly	
Joins	✅ In-memory only	⚡ On-disk + in-memory	
SQL syntax	❌ Not natively	✅ Built-in	
Parquet	✅ Okay	⚡ Super fast	
Memory usage	😓 High	⚡ Low / Lazy	
Workflow feel	🔪 Imperative	🎯 Declarative	